Evolución genética distribuida de autómatas de Langton generalizados: diseño e implementación en Rust, con evaluación empírica

Proyecto *Distributed Langton Ant*
código, datos y figuras reproducibles en el repositorio del proyecto

Octubre de 2026

Resumen

La hormiga de Langton es un autómata celular mínimo capaz de generar conducta emergente: tras unos diez mil pasos construye una "autopista" periódica. En este trabajo estudiamos una generalización del modelo (autómatas de Turing bidimensionales o *turmites*) y la evoluimos con un algoritmo genético distribuido de modelo de islas, implementado íntegramente en Rust y sin dependencias externas. Cada isla es un proceso independiente que se comunica con sus vecinas por TCP en un anillo, migrando asincrónicamente los dos mejores genomas cada cierto número de generaciones, mientras un monitor recolecta telemetría. La aptitud combina cuatro señales normalizadas —movilidad, densidad, complejidad morfológica y uso de color— y el espacio de búsqueda contiene $18^6 \approx 3.4 \times 10^7$ reglas para $S = 2$ estados y $C = 3$ colores. Sobre cinco semillas, el clúster de cuatro islas obtiene una aptitud final de 0.737 ± 0.012 frente a 0.714 ± 0.014 del modelo panmíctico y 0.714 ± 0.014 del secuencial, con una aceleración de $3.14\times$ respecto de este último y una eficiencia paralela del 78.4%. El portafolio heterogéneo redescubre cualitativamente cuatro familias de conducta (transportador, constructor, tejedor y equilibrado), incluida una cinta transportadora de período 9; sobre la regla clásica, el detector recupera la autopista de período 104. El artículo documenta el modelo, el algoritmo, el protocolo de red, la metodología experimental y todos los resultados numéricos; el código y los datos permiten regenerar cada tabla y figura.

Palabras clave: hormiga de Langton, turmite, autómatas celulares, algoritmos genéticos, computación distribuida, modelo de islas, Rust, vida artificial.

Abstract Langton's ant is a minimal cellular automaton that produces emergent behaviour: after roughly ten thousand steps it builds a periodic "highway". We study a generalization of the model (two-dimensional Turing machines, or *turmites*) and evolve it with a distributed island-model genetic algorithm written entirely in Rust with no external dependencies. Each island is an independent process that talks to its neighbours over TCP in a ring, asynchronously migrating its two best genomes every few generations while a monitor collects telemetry. Fitness combines four normalized signals—mobility, density, morphological complexity and colour usage—over a search space of $18^6 \approx 3.4 \times 10^7$ rules for $S = 2$ states and $C = 3$ colours. Across five seeds the four-island cluster reaches a final fitness of 0.737 ± 0.012, versus 0.714 ± 0.014 for the panmictic model and 0.714 ± 0.014 for the sequential one, with a $3.14\times$ speedup over the latter and 78.4% parallel efficiency. A heterogeneous portfolio rediscovers four qualitatively different behaviours (conveyor, builder, weaver and balanced), including a period-9 conveyor belt, while the detector recovers the classic period-104 highway of the baseline rule. We document the model, the algorithm, the wire protocol, the experimental methodology and every numerical result; code and data suffice to regenerate each table and figure.

Índice

1 Introducción **2**

2 Trabajo relacionado **3**

3 Modelo **4**
3.1 Autómata de Turing bidimensional 4
3.2 Codificación del genoma 4
3.3 Simulación con rejilla dispersa 4
3.4 Detección y verificación de autopistas 5

4 Función de aptitud **5**

5 Algoritmo genético y arquitectura distribuida **6**
5.1 Ciclo evolutivo 6
5.2 Modelo de islas asíncrono 6
5.3 Detalles de implementación 6

6 Metodología experimental **7**

7 Resultados **8**
7.1 Validación con la regla clásica 8
7.2 Convergencia en una sola isla 8
7.3 Comparación distribuida 9
7.4 Ablación del intervalo de migración 10
7.5 Portafolio heterogéneo 10
7.6 Rendimiento del simulador 11

8 Discusión **11**

9 Conclusiones y trabajo futuro **13**

A Reproducción **13**

1 Introducción

Los autómatas celulares son modelos de cómputo en los que una regla local, aplicada de forma sincrónica sobre una rejilla, produce patrones globales que a menudo no pueden deducirse de la regla misma [19]. La *hormiga de Langton*, propuesta por Christopher Langton como ejemplo mínimo de vida artificial [10], es uno de los casos más citados: una "hormiga" ocupa una celda de una rejilla binaria, lee el color, lo invierte, gira a la derecha si la celda era blanca y a la izquierda si era negra, y avanza. Con esa regla de dos líneas el sistema exhibe tres fases: simetría inicial, caos aparente y, alrededor del paso 10^4, una *autopista* periódica de 104 pasos que se desplaza indefinidamente en diagonal [5].

La generalización natural, introducida por el propio Langton, permite S estados internos y C colores de celda, y se conoce como *turmite* [5, 10]. El espacio de reglas crece muy rápido y la mayoría de sus puntos produce conductas triviales, periódicas o degeneradas; encontrar reglas "interesantes" a mano es inviable. Esto convierte al modelo en un banco de pruebas ideal para búsqueda evolutiva: la función de aptitud puede premiar propiedades observables de la trayectoria y del patrón, y el resultado es una regla de fácil lectura y verificación.

Este trabajo aborda dos preguntas. La primera es de *diseño*: ¿qué familias de conducta emergente pueden descubrirse con una aptitud multiobjetivo sencilla y un algoritmo genético

estándar? La segunda es de *ingeniería*: ¿qué se gana al distribuir la población en islas que intercambian individuos por red, en términos de tiempo, calidad y diversidad? Para responderlas construimos un sistema completo y lo evaluamos con un protocolo reproducible de cinco semillas.

Contribuciones.

1. Un simulador determinista de turmites con rejilla dispersa y sin límites artificiales, y un detector de autopistas que *verifica* la periodicidad re-simulando y comparando el patrón trasladado, en lugar de confiar sólo en la recurrencia del estado local.
2. Una aptitud multiobjetivo de cuatro componentes normalizadas (movilidad, densidad, complejidad de vecindades 3×3 y uso de color), con perfiles de pesos que inducen conductas cualitativamente distintas.
3. Un algoritmo genético de islas asíncrono sobre TCP: cuatro procesos, topología en anillo, migración de los dos mejores genomas, inyección selectiva y monitor de telemetría, sin dependencias externas.
4. Una implementación en Rust (unos 3 100 líneas) que incluye planificador de hilos con `std::thread::scope`, codificador PNG propio y una interfaz de línea de comandos con nueve subcomandos.
5. Una evaluación empírica con presupuestos iguales, cinco semillas, pruebas de permutación exactas y todos los datos crudos versionados junto al código.

El resto del artículo se organiza así: la Sección 2 revisa el trabajo relacionado; la Sección 3 define el modelo y el detector de autopistas; la Sección 4 describe la aptitud; la Sección 5 presenta el algoritmo y la arquitectura distribuida; la Sección 6 detalla la metodología; la Sección 7 reporta los resultados; la Sección 8 los discute y la Sección 9 concluye.

2 Trabajo relacionado

La hormiga y sus generalizaciones. Langton introdujo la hormiga como ejemplo de cómputo emergente en el borde del caos [10, 11]. Gale et al. [5] estudiaron las autopistas, conjeturaron que toda configuración finita acaba construyendo una y analizaron turmites con más estados y colores. La conjetura de la autopista sigue abierta para la regla clásica, aunque se sabe que la trayectoria es no acotada; para versiones generalizadas se han construido reglas universalmente computacionales y se ha demostrado que predecir la trayectoria es computacionalmente difícil [4]. Trabajos recientes siguen descubriendo familias de autopistas en espacios de reglas generalizados, lo que confirma que el modelo dista de estar agotado.

Evolución de autómatas celulares. La evolución de reglas celulares con algoritmos genéticos tiene una larga tradición; el trabajo de Mitchell et al. [14] sobre autómatas que realizan cómputo es el ejemplo canónico de cómo la selección puede descubrir reglas no triviales en un espacio discreto. Nuestro enfoque difiere en el objetivo—no buscamos una función computacional sino morfologías y trayectorias—y en la escala: reglas diminutas ($S{\cdot}C = 6$ entradas) y simulaciones cortas (10^4 pasos), lo que permite muchas evaluaciones.

Calidad y diversidad. La presión por diversidad conductual se inspira en la búsqueda de novedad [12]: en lugar de premiar sólo el mejor valor escalar, registramos cuántas conductas distintas coexisten. Nuestra métrica es más modesta (cuatro componentes cuantizadas) pero cumple la misma función de diagnóstico y se reporta en todas las tablas.

Algoritmos genéticos paralelos y distribuidos. El modelo de islas es una de las formas clásicas de paralelizar un algoritmo genético [1, 2, 16]. La migración periódica de individuos entre subpoblaciones equilibra exploración y explotación, y su efecto depende de la topología, del intervalo y del número de migrantes [18]. Frente a la alternativa panmíctica —una sola población evaluada en paralelo— el modelo de islas preserva diversidad genética y permite escalar a máquinas físicamente separadas. Aquí implementamos la variante asíncrona: no hay barrera entre generaciones y los mensajes se reciben cuando llegan, lo que evita sincronizar procesos lentos a costa de perder reproducibilidad bit a bit.

Rust. La implementación usa sólo la biblioteca estándar [9]: hilos con `scope`, canales `mpsc`, TCP y escritura de CSV. La ausencia de dependencias hace que el binario sea trivial de compilar y de auditar, propiedad valiosa para un experimento que aspira a ser reproducible [15].

3 Modelo

3.1 Autómata de Turing bidimensional

Un turmite se define por la tupla (S, C, δ), donde $S = \{0, \ldots, S-1\}$ es el conjunto de estados internos de la hormiga, $C = \{0, \ldots, C-1\}$ el de colores de celda y

$$\delta : S \times C \longrightarrow C \times \{\mathrm{L}, \mathrm{R}, \mathrm{U}\} \times S \tag{1}$$

es la tabla de reglas. En cada paso, con la hormiga en la posición (x, y), dirección $d \in \{\mathrm{N}, \mathrm{E}, \mathrm{S}, \mathrm{W}\}$ y estado s, se lee el color c de la celda, se escribe $\delta(s, c).c'$, se gira $\delta(s, c).t$ (90° a la izquierda, a la derecha o 180°), se avanza una celda en la nueva dirección y se adopta el estado $\delta(s, c).s'$. La regla clásica de Langton es el caso $S = 2$, $C = 2$ con

(s, c)	escritura, giro, estado	hex
$(0, 0)$	1, derecha, 0	`010100`
$(0, 1)$	0, izquierda, 0	`000000`
$(1, \cdot)$	no alcanzable	`000000`

es decir, el genoma `010100000000000000000000`. Para $S = 2$ y $C = 3$ el número de reglas es

$$|\mathcal{G}| = (C \cdot 3 \cdot S)^{S \cdot C} = 18^6 = 34\,012\,224, \tag{2}$$

un espacio lo bastante grande para que la búsqueda exhaustiva sea costosa y lo bastante pequeño para que un genoma quepa en 36 caracteres hexadecimales.

3.2 Codificación del genoma

Cada entrada de la tabla se serializa en tres bytes: color escrito, giro ($0 = \mathrm{L}, 1 = \mathrm{R}, 2 = \mathrm{U}$) y estado siguiente. Un genoma de $S \cdot C$ entradas ocupa $6SC$ caracteres hexadecimales; esta representación es la que viaja por la red y la que se escribe en los archivos de resultados, de modo que cualquier individuo puede re-evaluarse con un solo comando; el formato de los mensajes de migración se describe en la Sección 5.

3.3 Simulación con rejilla dispersa

La rejilla se implementa como un `HashMap` de bloques de 32×32 celdas; sólo se materializan los bloques visitados. Leer una celda no visitada devuelve el color 0 sin asignar memoria, y la huella es proporcional al área efectivamente explorada, no al desplazamiento. Esto permite simular autopistas que se alejan indefinidamente sin fijar un toro ni un límite arbitrario, algo esencial porque varias conductas óptimas son precisamente aquellas que escapan. La simulación es determinista: misma regla, mismas condiciones iniciales, misma traza.

Tabla 1: Perfiles de pesos. Cada perfil induce una presión evolutiva distinta y se usa para especializar islas del portafolio.

Perfil	w_m	w_ρ	w_κ	w_χ
`mover`	0.80	0.05	0.05	0.10
`builder`	0.05	0.75	0.10	0.10
`weaver`	0.20	0.20	0.50	0.10
`balanced`	0.50	0.25	0.15	0.10

3.4 Detección y verificación de autopistas

Decir que una regla "forma una autopista" requiere algo más que observar que la hormiga se repite. Definimos una autopista como una *periodicidad espacio-temporal*: existen un período p y una deriva constante $(\Delta x, \Delta y)$ tales que

1. la secuencia de estados locales (d_t, s_t) es periódica de período p;
2. el desplazamiento entre t y $t+p$ es el mismo $(\Delta x, \Delta y)$ para todo t en la ventana final;
3. el patrón construido en $t-p$, trasladado por $(\Delta x, \Delta y)$, coincide con el patrón en t en una ventana de radio 128 alrededor de la posición anterior de la hormiga.

La condición (3) es la que separa una autopista real de una coincidencia del estado local. Para verificarla se re-simula hasta $t-p$ y se compara celda a celda con el estado final trasladado. Los periodos candidatos se buscan probando la periodicidad de los últimos $4p$ símbolos para todo $p \le \min(T/3, 4096)$; se aceptan como mucho ocho candidatos y se exige coincidencia exacta y al menos 50 celdas comparadas. Con la regla clásica y $T = 120\,000$ el detector recupera $p = 104$, deriva $(-2, 2)$ y coincidencia 1.0 (Tabla 2), lo que valida el procedimiento.

4 Función de aptitud

Un genoma se evalúa simulando T pasos desde una celda vacía. Sea D el desplazamiento neto, A el área de la caja envolvente, N el número de celdas distintas visitadas y K el número de vecindades 3×3 distintas en el patrón. Definimos

$$m = \min\!\left(1, \frac{\|D\|_2}{T}\right) \qquad \text{movilidad,} \tag{3}$$

$$\rho = \min\!\left(1, \frac{N}{A}\right) \qquad \text{densidad,} \tag{4}$$

$$\kappa = \min\!\left(1, \frac{K}{512}\right) \qquad \text{complejidad,} \tag{5}$$

$$\chi = \frac{|\{c : \ c \text{ usado}\}|}{C} \qquad \text{color,} \tag{6}$$

y la aptitud escalar es

$$f = w_m m + w_\rho \rho + w_\kappa \kappa + w_\chi \chi, \qquad \textstyle\sum_i w_i = 1. \tag{7}$$

Las cuatro componentes están en $[0, 1]$, lo que hace que los pesos sean interpretables y las ablaciones directas. La suma ponderada es la forma más simple de escalarizar un problema multiobjetivo [3]; no pretende aproximar el frente de Pareto, sino ofrecer una familia de objetivos unidimensionales que podamos comparar. La Tabla 1 muestra los perfiles usados; el perfil *balanced* es el predeterminado.

La complejidad merece dos comentarios. Primero, se cuenta el número *absoluto* de vecindades distintas, no su fracción: así un patrón diminuto con pocos motivos no se considera complejo.

Segundo, el conteo es aproximado: se muestrean las celdas visitadas con paso 4 y se usa un filtro de Bloom de 2^{16} bits, lo que subestima ligeramente K por colisiones pero cuesta $O(N/4)$ en lugar de mantener un conjunto de patrones. La implementación devuelve el número de bits activos, de modo que el valor reportado es una cota inferior del número real de vecindades.

Además, una regla que visita menos de dos celdas recibe aptitud 0: sin esta guarda, reglas que no se mueven y no construyen nada podrían obtener densidad alta de forma espuria.

5 Algoritmo genético y arquitectura distribuida

5.1 Ciclo evolutivo

Cada isla mantiene una población de $P = 64$ genomas. El esquema es el algoritmo genético clásico [6, 8, 17]: generacional, con elitismo, torneo, cruce uniforme y mutación por campo.

1. evaluar los individuos pendientes (en paralelo con hasta `threads` hilos);
2. registrar estadísticas: mejor, media, mediana, peor, desviación, diversidad genotípica y conductual;
3. copiar los $\epsilon = 2$ mejores sin cambios;
4. generar el resto por torneo de tamaño $k = 3$, cruce uniforme por entrada de regla ($p = 0{,}5$ por entrada) y mutación por campo con probabilidad 0,03;
5. con probabilidad 0,01 reiniciar el hijo con un genoma aleatorio;
6. aplicar deduplicación: si la huella FNV-1a del hijo ya existe, se muta hasta cuatro veces y, si persiste, se reemplaza por uno aleatorio.

La población inicial contiene la regla clásica y $P/8 - 1$ mutaciones suaves de ella, lo que sesga el arranque hacia una región conocida sin impedir la exploración. Para $S = 2, C = 3$ el genoma tiene seis entradas, de modo que un hijo recibe en promedio $6 \cdot 3 \cdot 0{,}03 = 0{,}54$ mutaciones de campo: la presión es fina pero no despreciable.

La *diversidad conductual* se define como la fracción de individuos con una tupla (m, ρ, κ, χ) cuantizada a tres decimales distinta. No interviene en la selección (que es puramente por f), pero se registra en cada generación y se usa como diagnóstico de convergencia prematura.

5.2 Modelo de islas asíncrono

El clúster consta de $I = 4$ procesos, cada uno con su propia población de 64 individuos, conectados en anillo: la isla i envía a la isla $(i + 1) \bmod I$. Cada g generaciones ($g \in \{0, 5, 20\}$ en los experimentos) la isla ordena su población y envía los $M = 2$ mejores genomas como una única línea de texto. La recepción es *no bloqueante*: al final de cada generación se vacía el canal y los inmigrantes se inyectan reemplazando a los peores individuos no elite, sólo si su huella no existe ya en la población. No hay barrera global: una isla lenta no detiene a las demás, y por eso dos ejecuciones con la misma semilla pueden diferir en los detalles aunque compartan la semilla inicial.

La Figura 1 resume la arquitectura. Un quinto proceso, `monitor`, escucha los mensajes `TEL` y `END` y escribe dos CSV: la telemetría por generación y el resumen final por isla, que incluye mensajes y bytes enviados. El protocolo cabe en tres líneas:

```
MIG <emisor> <generacion> <hex>[,<hex>...]
TEL <isla> <gen> <mejor> <media> <diversidad> <evaluaciones> <ms>
END <isla> <mejor> <gen> <ms> <mensajes> <bytes>
```

5.3 Detalles de implementación

Los números aleatorios provienen de un generador xorshift [13] sembrado por isla. El binario `langton-ga` tiene nueve subcomandos: `sim`, `analyze`, `evolve`, `island`, `monitor`, `demo`, `bench`,

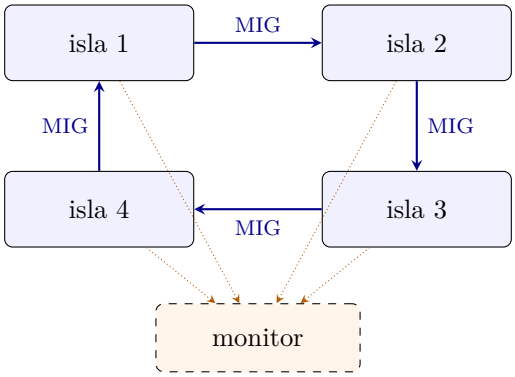


Figura 1: Arquitectura del clúster. Cuatro islas en anillo intercambian los dos mejores genomas; cada isla envía telemetría a un monitor independiente. Las flechas continuas son migración; las punteadas, telemetría.

`render` y `help`. Todo el código usa sólo `std`. La evaluación paralela dentro de una isla emplea `std::thread::scope` y divide la población en bloques contiguos, sin canales ni `Arc` en el camino crítico. La comunicación usa sockets TCP con timeouts de 500 ms y líneas terminadas en salto de línea, de modo que el protocolo se puede inspeccionar con `nc`. Las figuras se generan sin bibliotecas gráficas: un codificador PNG propio (bloques zlib almacenados, CRC-32 y Adler-32) escribe las imágenes que aparecen en este artículo.

El Listado 1 muestra la invocación mínima de un clúster de cuatro islas y la reproducción de un genoma concreto.

Listing 1: Invocación del clúster y re-evaluación de un genoma.

```
# cluster de 4 islas, 80 generaciones, migracion cada 5
./target/release/langton-ga demo --islands 4 --gens 80 --pop 64 \
    --steps 10000 --seed 1 --migration-interval 5 --migrants 2 \
    --states 2 --colors 3 --weights 0.20,0.20,0.50,0.10 \
    --base-port 8300 --out-dir results/distributed/m5_1

# re-evaluar el mejor genoma encontrado
./target/release/langton-ga analyze --states 2 --colors 3 --steps 60000 \
    --rule 02010000000102000102010102000101010O
```

6 Metodología experimental

Entorno. Todos los experimentos se ejecutaron en un Intel Core i5-8265U (4 núcleos físicos, 8 hilos lógicos) con 8 GB de RAM, Linux 7.2 y Rust 1.99 en modo `release`. Cada isla usa un solo hilo; el modelo panmíctico usa cuatro.

Presupuesto. Todos los métodos tienen el mismo presupuesto nominal: $4 \times 64 \times 80 = 20\,480$ individuos-generación, cada uno simulado $T = 10\,000$ pasos. El modelo secuencial y el panmíctico usan una población de 256 y 80 generaciones. Las evaluaciones efectivas son algo menores porque los élites se reutilizan sin re-simular: 20322 en secuencial y panmíctico y 19919 en el clúster (una diferencia del 2%). La única diferencia real es cómo se reparte el trabajo.

Semillas y réplicas. Cinco semillas por configuración $(1, \ldots, 5)$. En el clúster, la semilla s genera las semillas de isla $100s + i$, de modo que cada réplica arranca poblaciones distintas y la ejecución es repetible salvo por el orden de llegada de los mensajes.

Métricas. Se reportan: (i) la mejor aptitud final; (ii) el AUC de la curva de mejor aptitud, $\frac{1}{G}\sum_g f_g$, que premia la rapidez de convergencia; (iii) el tiempo hasta alcanzar $f = 0{,}70$; (iv) la

Tabla 2: Métricas de la regla clásica de Langton tras 120 000 pasos.

Métrica	Valor
Periodo de la autopista	104 pasos
Deriva por periodo	(-2, 2)
Velocidad de traslación	0.0272 celdas/paso
Coincidencia del patrón	1.0000
Celdas visitadas	24652
Revisitas	95348
Bounding box	2163×2135
Movilidad	0.0250
Densidad	0.0053
Complejidad	0.4824
Vecindades 3×3	247
Aptitud (perfil balanced)	0.186

diversidad conductual final; (v) el tiempo de pared; (vi) la aceleración $S_p = T_1/T_p$ y la eficiencia $E_p = S_p/p$; y (vii) el tráfico de control.

Estadística. Con cinco réplicas usamos pruebas de permutación pareadas exactas [7]: se enumeran los $2^5 = 32$ cambios de signo de las diferencias y se calcula la proporción con media absoluta al menos la observada. El mínimo p alcanzable es $2/32 = 0{,}0625$, por lo que el análisis es exploratorio; complementamos cada prueba con el tamaño del efecto y con el número de semillas en que un método supera a otro.

Reproducibilidad. El script `scripts/run_experiments.sh` ejecuta la suite completa y `scripts/make_report.py` agrega los CSV y regenera tablas, figuras y macros. Los resultados crudos están en `results/` y el PDF se compila con `latexmk`.

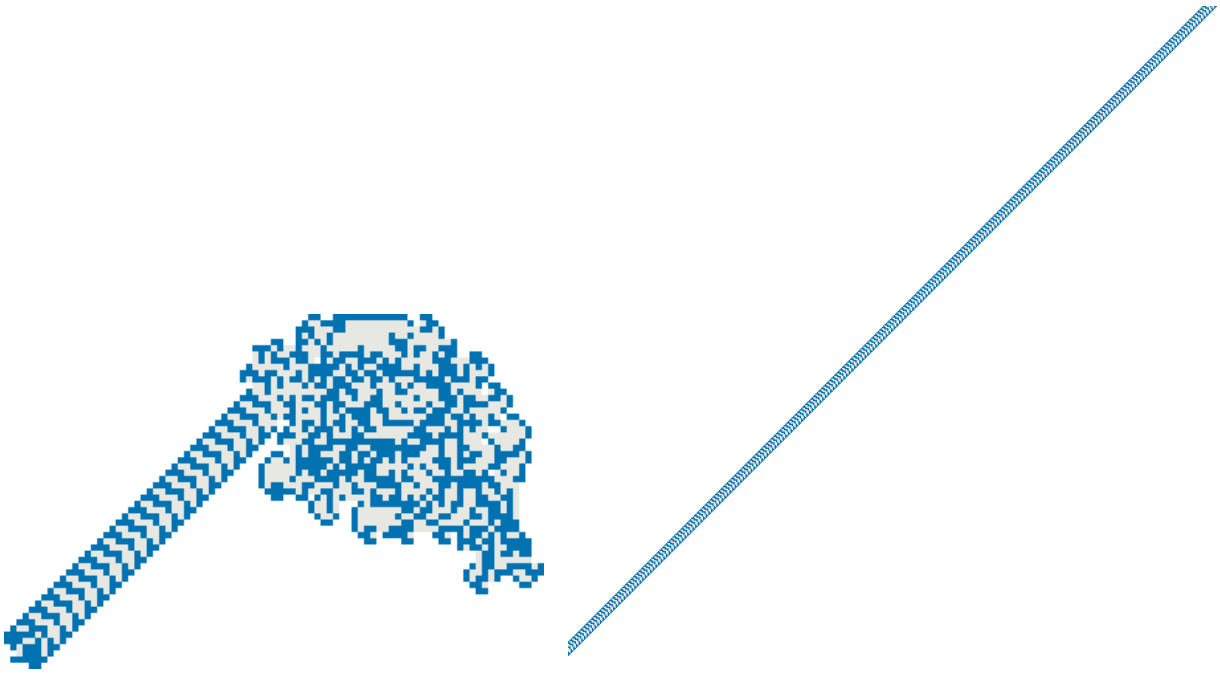
7 Resultados

7.1 Validación con la regla clásica

La Tabla 2 confirma que el simulador y el detector reproducen el comportamiento conocido. La autopista aparece con período 104 y deriva $(-2, 2)$, una velocidad de 0.0272 celdas por paso y coincidencia exacta del patrón trasladado (1.0000). La Figura 2 muestra el aspecto caótico a los 12 000 pasos y la autopista ya formada a los 120 000. Obsérvese la densidad extremadamente baja ($\rho = 0.0053$): la autopista es una huella fina dentro de una caja envolvente enorme, de 2163×2135 celdas. Este es exactamente el tipo de conducta que un objetivo de densidad puro penalizaría, y justifica que la aptitud combine varias señales.

7.2 Convergencia en una sola isla

Antes de distribuir la población conviene comprobar que el algoritmo genético converge por sí solo. La Figura 3 muestra la evolución con población 64 y 80 generaciones para dos perfiles. El perfil *weaver* alcanza 0.7041 ± 0.0444 (cuatro semillas por encima de 0,71 y una en 0,625), mientras que *balanced* se estanca en 0.5179 ± 0.0004. La diferencia no es de velocidad sino de óptimo: el perfil equilibrado converge rápidamente a una única regla densa y estable (desviación de 0,0004), y el perfil de complejidad encuentra patrones mejores pero con mucha más varianza entre semillas.



(a) 12 000 pasos: fase caótica. **(b)** 120 000 pasos: autopista.

Figura 2: Regla clásica. El color indica el número de visitas a cada celda.

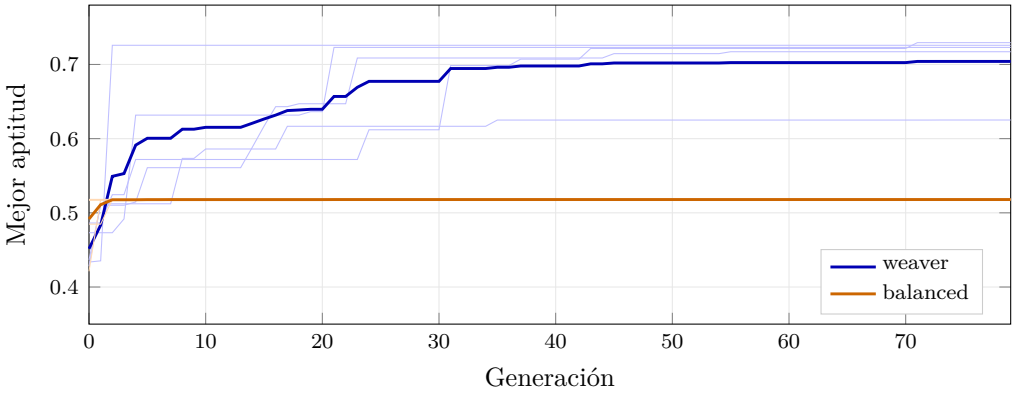


Figura 3: Convergencia de una isla de 64 individuos. Líneas finas: semillas; gruesas: media. El perfil *balanced* converge antes pero a una meseta más baja.

7.3 Comparación distribuida

La Tabla 3 compara los tres esquemas con el mismo presupuesto nominal. El clúster de islas es 3.14× más rápido que la versión secuencial (eficiencia 78.4% sobre cuatro procesos) y 2.41× más rápido que el panmíctico con cuatro hilos. El tiempo hasta alcanzar $f = 0{,}70$ resume la misma historia: 6.19 s en secuencial, 2.15 s en panmíctico y 1.26 s en el clúster.

El secuencial y el panmíctico obtienen cifras *idénticas* (0.714 ± 0.014 y 0.714 ± 0.014, AUC 0.683 y 0.683, diversidad 0.697 y 0.697) porque ejecutan exactamente la misma búsqueda: la misma semilla produce las mismas decisiones, y el paralelismo sólo cambia el tiempo. Es una comprobación útil de que el planificador de hilos no altera la trayectoria evolutiva.

El clúster mejora ligeramente la calidad: 0.737 ± 0.012 frente a 0.714 ± 0.014, un $+0{,}023$ (3,2%) que se observa en las cinco réplicas. Con cinco semillas la prueba de permutación pareada no puede bajar de $p = 0.0625$, de modo que el resultado debe leerse como una tendencia consistente y no como una diferencia confirmada. El AUC también es superior (0.711 frente a 0.683): el clúster no sólo llega un poco más lejos, sino que progresa antes. La diversidad conductual tardía

Tabla 3: Comparación de los tres esquemas de ejecución con el mismo presupuesto nominal ($T = 10\,000$). Media ± desviación sobre cinco semillas. La diversidad conductual es la media de las últimas 16 generaciones.

Modo	Tiempo (s)	Acel.	Evaluaciones	Mejor final	AUC	Div. conductual
Secuencial	18.98	1.00	20322	0.714 ± 0.014	0.683	0.697
Panmíctico	7.88	2.41	20322	0.714 ± 0.014	0.683	0.697
Islas (m=5)	6.05	3.14	19919	0.737 ± 0.012	0.711	0.899

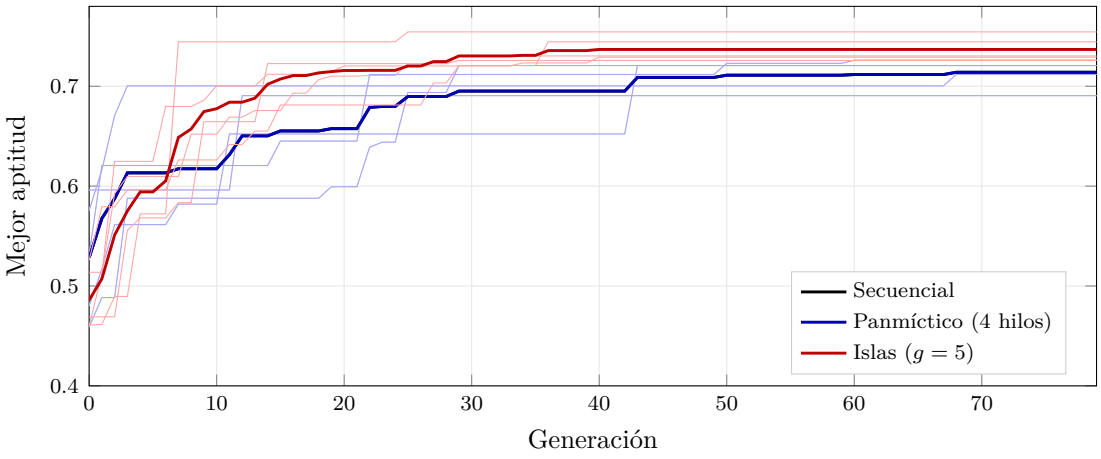


Figura 4: Convergencia de la mejor aptitud. Líneas finas: semillas individuales; líneas gruesas: media de cinco semillas.

es claramente mayor (0.899 frente a 0.697), en línea con la idea de que varias subpoblaciones independientes exploran cuencas distintas; en una de las réplicas secuenciales la población colapsa a una diversidad de 0,16, mientras que ninguna isla baja de 0,79. La Figura 4 muestra las curvas completas: las líneas finas son las cinco semillas y la gruesa su media.

7.4 Ablación del intervalo de migración

La Tabla 4 y la Figura 5 estudian el efecto del intervalo de migración. Sin migración ($g = 0$) las cuatro islas evolucionan de forma independiente; la diversidad conductual tardía es 0.869 y pasa a 0.899 con $g = 5$ y a 0.877 con $g = 20$. La mejor aptitud final es prácticamente la misma en los tres regímenes: 0.736 ± 0.008 sin migración, 0.737 ± 0.012 con $g = 5$ y 0.737 ± 0.012 con $g = 20$. Las diferencias no son significativas ($p = 1.0000$ entre $g = 0$ y $g = 5$; $p = 0.5000$ entre $g = 5$ y $g = 20$) y el tamaño del efecto es despreciable.

La explicación es estructural: el máximo del clúster es el máximo sobre cuatro islas, y el paisaje de aptitud tiene unos pocos atractores fuertes. Un puñado de reglas (0,7445, 0,7291, 0,7258) explica la mayoría de los finales de las quince ejecuciones, de modo que basta con que *una* isla encuentre el atractor para fijar el máximo; la migración no puede mejorarlo. Su efecto se observa, si acaso, en la diversidad ($p = 0.1875$, no significativo) y en el tráfico de control, que pasa de 14.2 KB sin migración a 19.0 KB con $g = 5$ (el resto es telemetría: un mensaje por isla y generación).

7.5 Portafolio heterogéneo

El experimento más ilustrativo cualitativamente es el portafolio: cuatro islas con perfiles de pesos distintos y sin migración, 60 generaciones. La Tabla 5 resume los mejores genomas y la Figura 6 muestra los patrones. Cada perfil encuentra una conducta diferente:

- **Mover** ($w_m = 0{,}80$) produce un deslizador diagonal: una autopista de período 8 con deriva

Tabla 4: Efecto del intervalo de migración g en el clúster de cuatro islas (0 = sin migración). Media ± desviación sobre cinco semillas; la diversidad es la media de las últimas 16 generaciones.

Intervalo	Mejor final	AUC	Div. conductual	Tiempo (s)	Control (KB)
0	0.736 ± 0.008	0.703	0.869	5.51	14.2
5	0.737 ± 0.012	0.711	0.899	6.05	19.0
20	0.737 ± 0.012	0.702	0.877	5.95	15.2

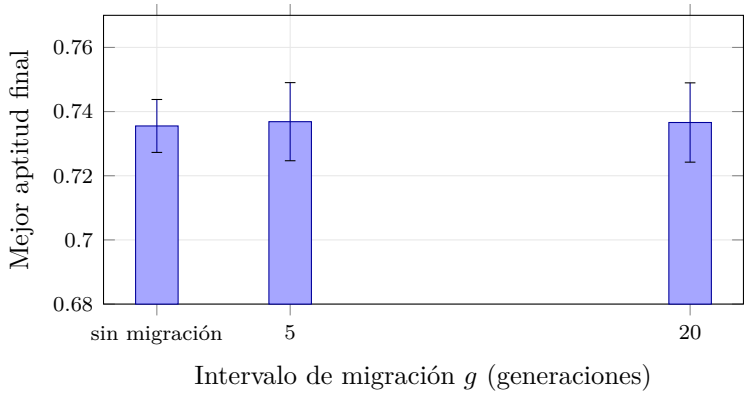


Figura 5: Aptitud final media (± desviación) según el intervalo de migración.

$(4, -4)$, es decir 0.707 celdas por paso, y sólo tres vecindades 3×3 distintas. Es movilidad pura.

- **Builder** ($w_\rho = 0{,}75$) construye una banda densa de dos filas que llena el 99,9% de su caja envolvente y se desplaza lentamente (período 10, deriva $(0, 2)$). Al igual que la cinta, es una estructura cuasi unidimensional: la densidad se maximiza "adelgazando" el patrón hasta una franja.
- **Weaver** ($w_\kappa = 0{,}50$) genera un patrón ramificado y estático con 1548 vecindades 3×3 distintas y movilidad prácticamente nula.
- **Balanced** descubre una cinta transportadora de período 9 y deriva $(0, 3)$: un caso intermedio entre transporte y estructura, con movilidad 0,333 y densidad alta.

Los cuatro genomas tienen aptitud alta *bajo su propio perfil* (entre 0.484 y 0.861), pero al re-evaluarlos con el perfil *weaver* solo el tejedor conserva un valor alto (0,689): el constructor cae a 0,345, el equilibrado a 0,336 y el deslizador a 0,211. Los perfiles no son variaciones cosméticas del mismo objetivo: inducen regiones diferentes del espacio de reglas.

7.6 Rendimiento del simulador

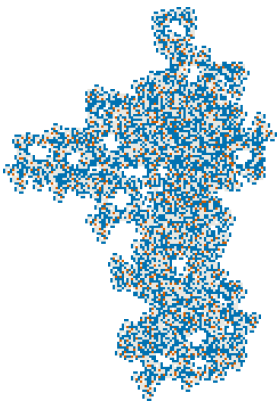
La Tabla 6 resume el costo de una evaluación aislada ($T = 20\,000$, 100 iteraciones). El simulador procesa 12.13 millones de pasos por segundo en un núcleo, lo que explica que una generación de 64 individuos con $T = 10^4$ tarde del orden de 0,05 s en un núcleo. La rejilla dispersa y la ausencia de asignaciones en el camino caliente son responsables de esta cifra.

8 Discusión

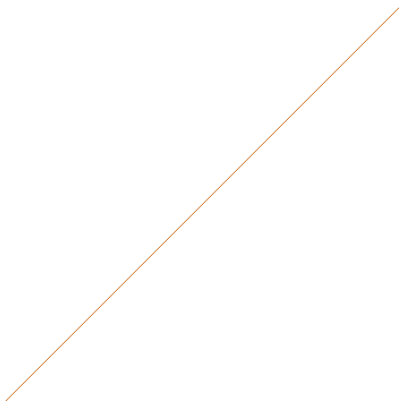
¿Por qué el clúster solo mejora ligeramente? El clúster reparte el presupuesto en cuatro subpoblaciones de 64 durante 80 generaciones, mientras que el panmíctico concentra 256 individuos en una sola bolsa genética. La migración ocurre solo 16 veces ($g = 5$), de modo que cada isla explora la mayor parte del tiempo por su cuenta; el efecto observado es una mejora pequeña

Tabla 5: Portafolio heterogéneo: mejor genoma por perfil tras 60 generaciones. La última columna indica período y deriva cuando el análisis detecta una autopista.

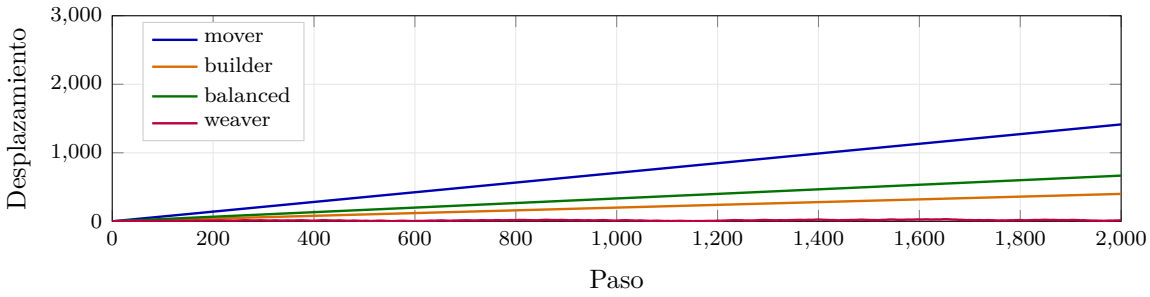
Perfil	Aptitud	Apt. weaver	Movilidad	Densidad	Compl.	Vec.	Per. / deriva
mover	0.633	0.211	0.7071	0.000	0.006	3	8 / (4,-4)
builder	0.861	0.345	0.2000	1.000	0.010	5	10 / (0,2)
weaver	0.707	0.689	0.0014	0.445	1.000	1548	—
balanced	0.484	0.336	0.3333	1.000	0.006	3	9 / (0,3)



(a) Tejedor ($w_\kappa = 0{,}50$)



(b) Deslizador diagonal ($w_m = 0{,}80$)



(c) Desplazamiento acumulado: transporte lineal frente a patrón localizado

Figura 6: Conductas descubiertas por el portafolio heterogéneo. (a,b) Patrones finales; el color codifica el número de visitas. (c) Distancia al origen en función del tiempo para las cuatro reglas.

pero consistente (+3,2%, cinco de cinco réplicas) y una diversidad tardía mucho mayor. La interpretación razonable es que las islas conservan varias cuencas de atracción y que el máximo sobre cuatro poblaciones aprovecha esa exploración, mientras que la población panmíctica converge más rápido a un único atractor. Con un paisaje tan pequeño y solo cinco semillas no es posible separar este efecto del azar; para ello harían falta más réplicas o un espacio de búsqueda mayor. La ventaja más sólida del clúster es de *ingeniería*: el mismo resultado en un tercio del tiempo y la posibilidad de crecer a varias máquinas.

Diversidad y especialización. El portafolio muestra que la diversidad no es un adorno: con cuatro perfiles se obtienen cuatro morfologías que un único objetivo escalar no visitaría. Esto conecta con la literatura de novedad y calidad-diversidad [12] y sugiere una extensión natural: usar la diversidad conductual como criterio de selección o de archivo, en lugar de sólo como métrica.

La aptitud como hipótesis. Los resultados dependen fuertemente de los pesos. El perfil *balanced* aumenta la movilidad y la complejidad a costa de la densidad, y por eso redescubre una cinta; el perfil *weaver* prioriza complejidad y produce patrones estáticos. No hay una aptitud

Tabla 6: Rendimiento de la evaluación (un núcleo, $T = 20\,000$, 100 repeticiones).

Métrica	Valor
Evaluaciones por segundo	606.6
Pasos de simulación por segundo	12 131 175
Millones de pasos por segundo	12.13
Celdas visitadas por segundo	2 120 529

"correcta": hay una familia de hipótesis sobre qué hace interesante a una regla, y el marco permite contrastarlas con el mismo presupuesto.

Amenazas a la validez. (i) Cinco semillas dan una potencia estadística baja: el p mínimo de una prueba de permutación pareada es 0,0625, insuficiente para el umbral convencional de 0,05. Reportamos tamaños de efecto y direcciones, no sólo valores p. (ii) Los experimentos se hicieron en una sola máquina con cuatro núcleos físicos; la aceleración no refleja latencias de red reales. (iii) El tráfico de control reportado incluye la telemetría (un mensaje por generación), que domina el total; el payload de migración es de $\sim$72 bytes por evento. (iv) La complejidad es una estimación por filtro de Bloom. (v) Las ejecuciones del clúster no son reproducibles bit a bit por la naturaleza asíncrona de la red; sí lo son las locales. (vi) El detector de autopistas exige coincidencia exacta en una ventana de radio 128; patrones más grandes podrían escaparse. (vii) Las islas arrancan con semillas derivadas ($100s + i$) distintas de las del secuencial y el panmíctico, de modo que la comparación emparejada por índice de réplica no usa números aleatorios comunes; los tamaños de efecto deben interpretarse con cautela.

9 Conclusiones y trabajo futuro

Construimos un sistema completo —modelo, simulador, aptitud, algoritmo genético, clúster TCP, monitor, figuras y suite experimental— para evolucionar turmites de Langton y lo evaluamos con un protocolo reproducible. El clúster de islas iguala o mejora ligeramente la calidad del panmíctico con una aceleración de 3.14× sobre la versión secuencial y una eficiencia del 78.4%, y el portafolio heterogéneo redescubre cuatro familias de conducta, incluida una cinta transportadora de período 9; la autopista clásica de período 104 se usa como validación del detector. El código, los datos crudos y los scripts permiten regenerar cada tabla y figura.

El trabajo futuro se orienta en cuatro direcciones: (i) selección por diversidad conductual o archivo de novedad, para evitar la convergencia de las islas; (ii) migración adaptativa, con intervalo y número de migrantes dependientes de la heterogeneidad medida; (iii) topologías distintas del anillo (grafo completo, malla, *gossip*) y comparación sistemática; y (iv) ejecución en varias máquinas con fallos y latencias reales, que es donde el modelo distribuido debería mostrar su ventaja.

A Reproducción

Listing 2: Secuencia completa de reproducción.

```
cargo build --release
cargo test --release                    # 22 pruebas
./scripts/run_experiments.sh            # resultados crudos
python3 scripts/make_report.py          # tablas, macros y figuras
cd paper && latexmk -pdf main.tex       # este documento
```

La correspondencia entre artefactos y secciones es la siguiente: `results/baseline/` alimenta la Tabla 2 y la Figura 2; `results/distributed/` alimenta las Tablas 3–4 y las Figuras 4–5; `results/portfolio/` y `results/gallery/` alimentan la Tabla 5 y la Figura 6. Los CSV agregados están en `paper/data/`, las tablas en `paper/tables/` y las figuras en `paper/figs/`.

Referencias

[1] Enrique Alba and Marco Tomassini. Parallelism and evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 6(5):443–462, 2002. doi: 10.1109/TEVC.2002.800880.

[2] Erick Cantú-Paz. A survey of parallel genetic algorithms. *Calculateurs Parallèles, Réseaux et Systèmes Répartis*, 10(2):141–171, 1998.

[3] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, Chichester, UK, 2001.

[4] Anahí Gajardo, Andrés Moreira, and Eric Goles. Complexity of Langton's ant. *Discrete Applied Mathematics*, 117(1–3):41–50, 2002. doi: 10.1016/S0166-218X(00)00334-6.

[5] David Gale, Jim Propp, Scott Sutherland, and Serge Troubetzkoy. Further travels with my ant. *The Mathematical Intelligencer*, 17(3):48–56, 1995. doi: 10.1007/BF03024370.

[6] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989.

[7] Phillip I. Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, New York, 3 edition, 2005. doi: 10.1007/b138696.

[8] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.

[9] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2 edition, 2023.

[10] Christopher G. Langton. Studying artificial life with cellular automata. *Physica D: Nonlinear Phenomena*, 22(1–3):120–149, 1986. doi: 10.1016/0167-2789(86)90237-X.

[11] Christopher G. Langton. Computation at the edge of chaos: Phase transitions and emergent computation. *Physica D: Nonlinear Phenomena*, 42(1–3):12–37, 1990. doi: 10.1016/0167-2789(90)90064-V.

[12] Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[13] George Marsaglia. Xorshift RNGs. *Journal of Statistical Software*, 8(14):1–6, 2003. doi: 10.18637/jss.v008.i14.

[14] Melanie Mitchell, Peter T. Hraber, and James P. Crutchfield. Revisiting the edge of chaos: Evolving cellular automata to perform computations. *Complex Systems*, 7(2):89–130, 1993.

[15] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

[16] Reiko Tanese. Distributed genetic algorithms. In *Proceedings of the Third International Conference on Genetic Algorithms*, pages 434–439, San Mateo, CA, 1989. Morgan Kaufmann.

[17] Darrell Whitley. A genetic algorithm tutorial. *Statistics and Computing*, 4(2):65–85, 1994. doi: 10.1007/BF00175354.

[18] Darrell Whitley, Soraya Rana, and Robert B. Heckendorn. The island model genetic algorithm: On separability, population size and convergence. *Journal of Computing and Information Technology*, 7 (1):33–47, 1998.

[19] Stephen Wolfram. *A New Kind of Science*. Wolfram Media, Champaign, IL, 2002.